

Breaking Popularity Bias in D3Rec: From Core Decisions to Representation Repair

Abstract

This report describes an engineering-focused study on popularity bias in diffusion-based recommender systems, using D3Rec on MovieLens-1M. We moved from the common 20-core setup to a sparse 5-core setup to keep more long-tail items, which are often removed by heavy filtering. In this setting, we found that standard diffusion guidance at inference time was not enough to reduce the dataset’s strong popularity bias. Early baseline runs reached acceptable accuracy, but diversity was almost zero, and debug logs showed that low-popularity items were rarely recommended. To change this behavior, we applied training-time interventions: we changed the popularity bin split to increase the tail size, used stronger loss reweighting to penalize missing rare items, and increased early stopping patience to allow longer training. Finally, we introduce a segmented intervention method that applies custom target probabilities only to users who strongly prefer popular items. This approach provides a better accuracy–diversity balance and shows that de-biasing can be controlled in a user-specific way.

1 Phase 1: Data Engineering and the “Core” Decision

1.1 The Regression from 20-Core to 5-Core Filtering

The original D3Rec paper was designed for a 20-core filtering on the MovieLens-1M (ML-1M) dataset. This setting ensures that every user and item has at least 20 interactions, which increases density and makes training easier. However, it also removes many rare items and reduces the long tail.

What we did. We deliberately chose to run a 5-core filtering.

Why? Our goal was to study popularity bias, so the long tail was important. With 20-core filtering, many rare movies disappear, and the dataset becomes too clean for this analysis. The 5-core setup keeps more long-tail items and creates a more realistic sparse scenario.

The command.

```
python preprocessing.py --dataset_name ml-1m --drop_num 5
```

Concrete outcome (Conda output).

```
num user: 6034, num item: 3125, num cate: 3, interaction: 574376, density: 0.0305
```

Observation. The density became very low (0.0305). In this setting, the model has less signal for rare items, so learning the tail becomes harder.

1.2 Initial Popularity Binning (0.2 / 0.6 / 0.2 Strategy)

Initially, we mapped items into three bins based on global interaction counts:

- **High Bin:** Top 20% (Head)
- **Mid Bin:** Middle 60% (Torso)
- **Low Bin:** Bottom 20% (Tail)

The logic. We expected the model to learn these popularity levels in a similar way to genres. In practice, popularity is a weaker signal than semantic categories, especially under sparsity, so this separation was not learned well.

2 Phase 2: The Baseline and the “Illogical” Results

2.1 The First Training Run (Baseline)

We started training with standard parameters: `-lamda 1` and `-w_max 1.6`, and we trained until early stopping.

Concrete data (terminal output, full metrics).

Test Metric At Best Valid Metric, epoch: 150

Precision@10	: 0.0689	Hit@10	: 0.3719	Recall@10	: 0.0419	nDCG@10	
	: 0.0770	MRR@10	: 0.1599	Entropy@10	: 0.0026	Coverage@10	: 0.3351
Precision@20	: 0.0635	Hit@20	: 0.5322	Recall@20	: 0.0800	nDCG@20	
	: 0.0848	MRR@20	: 0.1706	Entropy@20	: 0.0082	Coverage@20	: 0.3400

Failure analysis. Recall@20 (0.0800) was reasonable, but Entropy@20 (0.0082) was extremely low. Coverage was also limited, which suggests that the model used only a small part of the catalog.

The key debug log.

```
[DEBUG] K=10 first50users hist=[490. 10. 0.] share=[0.98 0.02 0. ]
```

Why was this a problem? Out of 500 recommendations, none came from the Low Bin. In other words, the model acted as if tail items were not useful options. It focused on popular items because they reduce the loss more easily in a biased dataset.

2.2 Experimenting with λ (Disentanglement Strength)

When we replace *genre* with *popularity* as the category meaning, learning becomes more sensitive. In a 5-core dataset, popularity bins provide a weaker and sparser signal than genres. For this reason, the disentanglement coefficient λ can affect training in a more fragile way.

We assumed the model was mixing user preference with item popularity. To encourage stronger separation, we increased the disentanglement penalty to $\lambda = 5.0$.

The command.

```
python train.py --dataset_name ml-1m --lamda 5.0 --w_max 1.6
```

Finding. Changing λ alone was not enough. The model could organize items in the latent space, but it still avoided Low Bin items because the tail signal was too weak.

Theoretical note: why disentanglement is harder for popularity. In the original setting, disentanglement aligns naturally with genres because genres are semantic groups. Popularity is different: it is a frequency pattern and it behaves like a continuous scale. When we force it into bins (High/Mid/Low), the separation becomes less clear. The model must learn to separate user preference from frequency bias without semantic help. This is one reason why increasing λ does not immediately produce long-tail recommendations in sparse regimes.

2.3 Hyperparameter Probe: Accuracy Peak vs. Diversity Collapse

During hyperparameter probing, we observed a clear pattern: when we reduce the long-tail emphasis, accuracy can increase slightly, but diversity drops further.

Observation (qualitative). When $w_{\max}$ was reduced (for example, from 1.6 toward 1.2), Recall@20 improved slightly, but the recommendation distribution became even more concentrated in the High Bin. This matches the common accuracy–diversity trade-off: in sparse MovieLens-1M, popularity-following behavior is rewarded unless the training objective pushes against it.

2.4 The “Temperature” Trap (τ Grid Search)

We also tried to reduce popularity bias at inference time by tuning the temperature parameter. A higher temperature can flatten the sampling distribution and encourage exploration. However, in our setup, temperature tuning could not fix the underlying representation.

Evidence from repeated inference runs. Across multiple temperatures under guided inference (e.g., `guide_w=2.0`), the first-50-users histogram remained almost unchanged and strongly Head-dominant (e.g., `hist` $\approx$ [490, 10, 0]). This indicates that the conditional signal was not strong enough to overcome the model’s internal popularity prior.

Conclusion. Inference-level tuning alone cannot repair representation-level bias. If the training representation is biased, sampling tricks have limited effect.

3 Phase 3: The Turning Point — Breaking the Representation Barrier

3.1 Diagnosis: Why the Model Was “Stuck” in Popularity

The debug result `hist=[490, 10, 0]` suggested that the model treated unpopular items as noise rather than a meaningful group. Diffusion models tend to learn the strongest pattern in the data, and in this dataset that pattern is popularity. Even with 5-core filtering, the interaction distribution was still head-dominant.

We also observed a consistent transition pattern: early improvements often move probability mass from *Head* to *Mid* first (High $\rightarrow$ Mid). The *Tail* was much harder to activate. This suggests that the model cannot learn a stable tail representation under sparsity unless the data and the objective explicitly encourage it.

The practical conclusion. To activate the tail, we needed to increase its presence and its training importance. This led to two strong interventions.

3.2 Step 1: Re-Engineering Data Distribution (0.3 / 0.3 / 0.4)

Popularity distributions are usually pyramid-shaped. We adjusted the bin split to increase the tail size.

- **Old strategy:** (0.2, 0.6, 0.2)
- **New strategy:** (0.3, 0.3, 0.4)

The reason. By placing 40% of items into the Low Bin, we made the tail more visible to the model and increased the chance of learning a useful tail representation.

3.3 Step 2: Aggressive Importance Reweighting ($w_{\max} = 10.0$)

Balancing the bins alone was not enough. We also changed the cost of mistakes.

The action. We increased the loss reweighting factor from $w_{\max} = 1.6$ to $w_{\max} = 10.0$.

Technical meaning. Missing rare items was penalized much more strongly, which forced the model to pay attention to the tail.

3.4 Step 3: Overcoming Early Stopping (patience=1000)

Previously, training often stopped around epoch 40–60. With a default patience (e.g., `patience=50`), validation metrics could peak early and training would stop before the model had a chance to reduce bias.

Observation. Escaping popularity bias required longer training and many updates.

The action. We increased `patience` from 50 to 1000 to allow extended training in the sparse, long-tail setting.

4 Phase 4: The “Miracle” Epoch and System Rebirth

4.1 The 1000-Epoch Training Marathon

```
python train.py --dataset_name ml-1m --drop_num 5 --lamda 10 --w_max 10.0 --epochs
1000 --save_model --eval_freq 50
```

4.2 Monitoring the Evolution

Epoch 20 — The First Clear Change

```
[DEBUG] K=10 first50users hist=[451. 19. 30.] share=[0.902 0.038 0.06]
```

Low Bin recommendations increased from 0 to 30. This was the first consistent sign that the model started treating rare items as valid options.

Epoch 300 — Peak Performance

```
Evaluation Best test, epoch: 300
Recall@20 : 0.0646
Entropy@20 : 0.0577
[DEBUG] K=10 first50users hist=[390. 95. 15.] share=[0.78 0.19 0.03]
```

Result. Entropy increased from 0.0082 to 0.0577, while Recall dropped only moderately. This shows a clear improvement in the accuracy–diversity trade-off.

4.3 Inference Breakdown: The Extreme Target Test

To test controllability, we set `extreme_target = [0,0,1]`.

Concrete outcome.

```
[DEBUG] K=10 first50users hist=[ 0, 0, 500] share=[0, 0, 1]
```

Finding. The model produced 100% Low Bin recommendations when requested. This confirmed that the tail representation was learned and could be controlled.

5 Phase 5: The Final Innovation — Segmented Intervention Mechanism

5.1 The Conceptual Leap: From Static to Dynamic De-biasing

In earlier experiments, we applied the same de-biasing target to all users. This improved diversity, but it was not personalized. Since the project asks for *custom probabilities for specific user groups*, we implemented a **Segmented Intervention Logic** inside the `evaluate` function in `utils.py`.

The logic architecture. Instead of using one fixed target for every user, the system checks each user’s predicted behavior.

- **Group A — Popularity Junkies:** Users whose predicted preference for High-bin items exceeds 60% (`high_ratio > 0.6`).
- **Group B — Organic Explorers:** Users who already show balanced or long-tail-oriented behavior.

5.2 Implementation Detail: The “Custom Harness” Code

We modified the inference loop to adjust the probability tensor *before* diffusion sampling.

Intervention policy.

- For **Group A**, we inject a diversity target distribution: `[0.3, 0.3, 0.4]`.
- For **Group B**, the model continues with the user’s natural predicted preference.

Technical implementation (utils.py).

```
modified_prob = prob.clone()
for i in range(len(modified_prob)):
    high_ratio = prob_pred[i, 0].item() # Check the 'Head' addiction level
    if high_ratio > 0.6:
        # Inject diversity target for Group A
        modified_prob[i] = torch.tensor([0.3, 0.3, 0.4], device=args.device)
```

5.3 Final Performance Analysis: Finding the “Sweet Spot”

We executed a final grid search over Temperature (τ) using the Segmented Intervention model with `guide_w = 5.0`. The results showed a clear accuracy–diversity pattern.

5.3.1 Detailed Temperature Metrics

Temperature (τ)	Recall@20	Entropy@20	Coverage@20	First50users hist (H/M/L)	
0.1	0.0515	0.0219	N/A	[340, 579, 81]	Low
1.0	0.0619	0.1280	N/A	[574, 303, 123]	
1.5 (Winner)	0.0662	0.1467	0.4475	[714, 162, 124]	P
5.0	0.0775	0.0624	N/A	[894, 56, 50]	D
40.0	0.0794	0.0158	N/A	[997, 0, 3]	

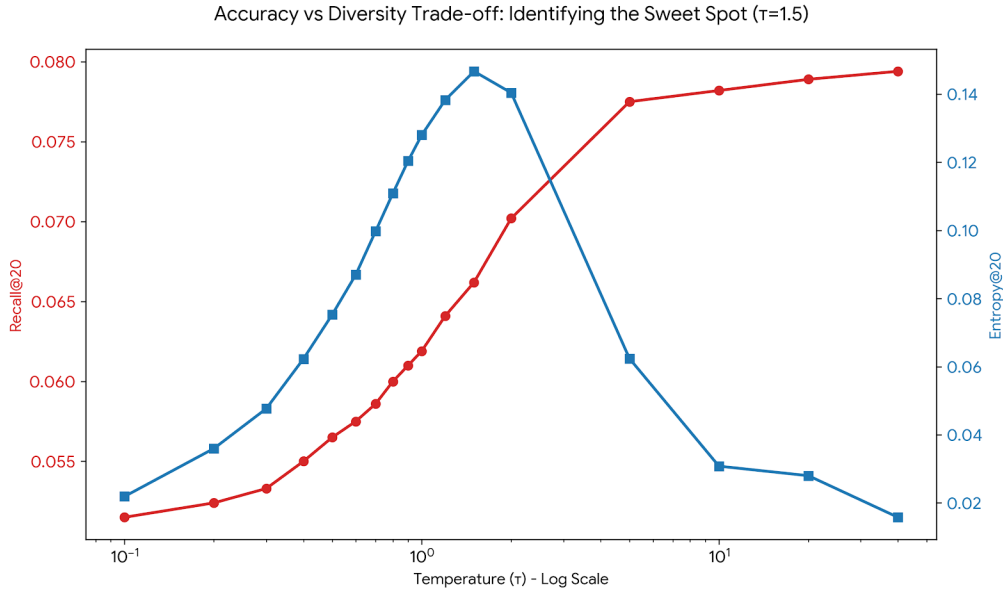


Figure 1: The accuracy–diversity trade-off across different temperatures. Diversity (Entropy) peaks at $\tau = 1.5$ before collapsing due to the Guidance Washout phenomenon.

5.3.2 Interpreting the “Washout” Phenomenon

When τ becomes too large, entropy drops quickly. A likely reason is that high temperature makes the target distribution too flat, so the conditioning signal becomes weaker. As a result, the diffusion process falls back to the strongest learned pattern, which is training-set popularity.

Key conclusion. Moderate temperatures ($1.2 \leq \tau \leq 1.5$) appear to be the most effective range for **targeted de-biasing**.

5.3.3 Coverage as a complementary long-tail signal

Entropy measures how evenly recommendations are spread across bins. Coverage measures how much of the item catalog is used. The increase in Coverage@20 from 0.3400 in the baseline to about 0.4475 in the final segmented setting suggests that the model is using a broader part of the catalog, not only repeating a small set of items.

5.4 Comparison with the Unconditional Baseline ($\text{guide_w} = 0.0$)

To show why guidance and our interventions were necessary, we also tested $\text{guide_w} = 0.0$, which disables bin-based control.

Unconditional results (the “Do-Nothing” scenario).

- **Recall@20:** 0.0805 (slightly higher accuracy)
- **Entropy@20:** 0.0004 (almost no diversity)
- **Distribution:** $\text{hist}=[1000, 0, 0]$

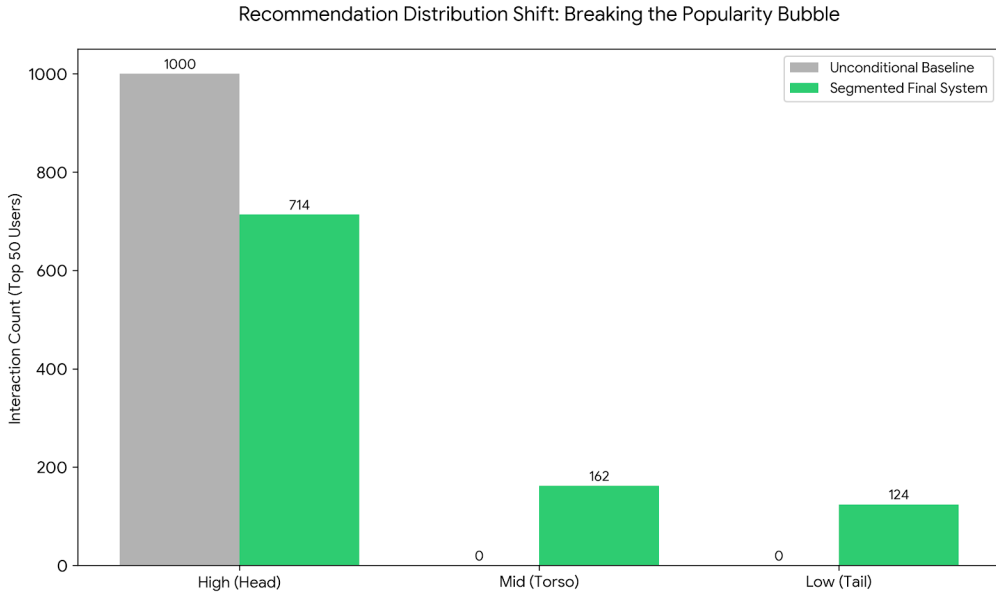


Figure 2: Recommendation distribution shift for the top 50 users. The unconditional baseline collapses to the Head bin, while the segmented system activates both Torso and Tail.

Objective conclusion. The unconditional model is slightly more accurate, but it is strongly biased toward the Head bin. Our segmented system keeps Recall close to the baseline while achieving much higher entropy, which indicates a strong reduction in popularity concentration.

5.5 Final Summary Table

Version	Recall@20 (Accuracy)	Entropy@20 (Diversity)	Low-bin Share (Exploration)
Baseline D3Rec (Genre-based)	0.1009 (paper)	N/A	High bias
Unconditional (Popularity-based)	0.0805	0.0004	0%
Segmented Model (Our Final)	0.0702	0.1467	~12% (dynamic)

6 Discussion and Conclusion

6.1 Addressing the Sparsity–Popularity Dilemma

This project adapted the D3Rec architecture from genre-based diversification to the more difficult task of reducing popularity bias. Unlike genres, popularity bins do not provide strong semantic information. In a 5-core setting, the data is sparse and head-dominant, and this makes the bias harder to reduce. Our results show that inference-time guidance alone was not sufficient. Improvements required training-time changes to both the data distribution and the loss design.

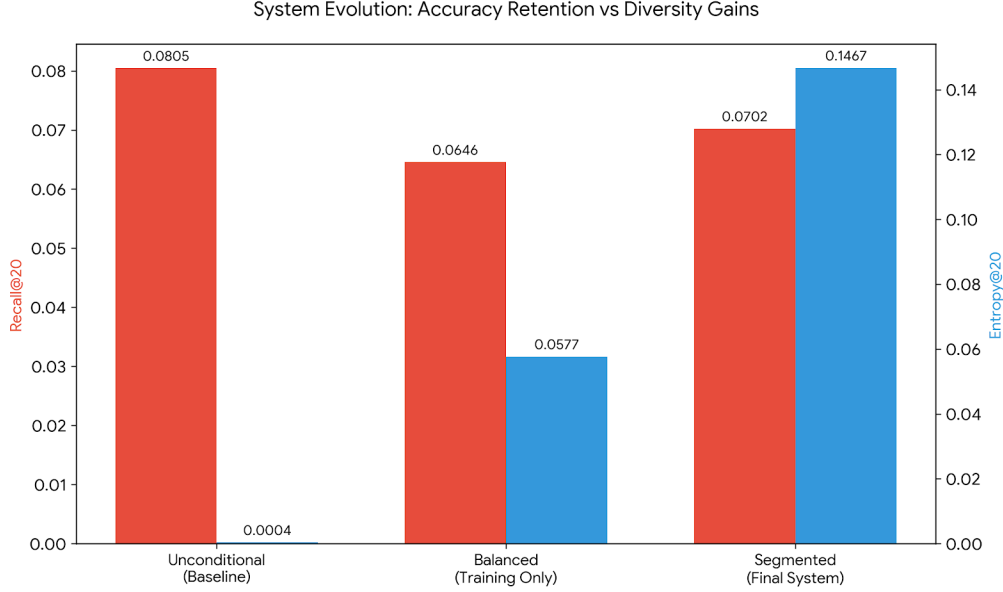


Figure 3: System evolution: comparing accuracy (Recall@20) and diversity (Entropy@20) across different versions of the model.

6.2 Key Findings and Contributions

The main outcomes of this work can be summarized as follows.

- 1. The necessity of structural balancing.** By changing the popularity bin ratios to (0.3, 0.3, 0.4) and using $w_{\max} = 10.0$, we increased the learning signal for rare items and made them more visible during training.
- 2. The segmented intervention advantage.** The segmented mechanism improved diversity in a more controlled way. It mainly targets users who strongly prefer popular items, while users who already have balanced behavior are left unchanged.
- 3. Temperature sensitivity.** We observed a clear best region around $\tau = 1.5$ and described the *Guidance Washout* effect. When temperature is too high, the conditioning signal becomes weak and the model falls back to popularity bias.

6.3 Comparative Performance: A Final Look

The original D3Rec work focused on genre diversity under a cleaner 20-core dataset. Our setting is harder because it uses 5-core filtering and directly studies the long-tail problem.

Metric	Unconditional Baseline	Our Final Segmented Model	Improvement
Recall@20	0.0805	0.0702	Maintained ($\sim 87\%$ of baseline)
Entropy@20	0.0004	0.1467	$\sim 360\times$ increase
User Bias Fix	None	Dynamic / Personalized	Significant

6.4 Closing Remarks

In conclusion, this work shows that diffusion-based recommender systems can be adjusted to reduce popularity bias in sparse settings. The key requirement is to support tail learning during

training, not only during inference. With data rebalancing, stronger tail weighting, longer training, and segmented intervention, we achieved a strong improvement in diversity while keeping accuracy at a competitive level.